MemSolve: From Sequential Memory Updates to Global Equilibrium

Zhaoyang Wang
zhaoy@uir.edu.cn

Working draft — October 20, 2026

Abstract

Gated Delta Networks (GDNs) learn and refine associations by writing prediction errors back into memory. This error-correcting mechanism has advanced causal sequence modeling. In bidirectional settings, the model can access the entire context. It can therefore build a shared memory from all positions. Yet extending sequential error correction to this setting can be costly. The direct full-attention DeltaNet formulation studied in LION requires a sequence-sized matrix inverse. Its cubic cost undermines the efficiency of a compact memory. We propose MemSolve, which constructs memory by directly solving a joint error-correction problem over the full context. Associations from all positions jointly determine a compact equilibrium state. Each position then reads context from this shared memory. Memory writes and reads depend on the entire context. This allows information on both sides of a position to interact. Locally parameterized bidirectional scans with linear merging cannot express all such interactions. We prove that the equilibrium is unique and that the readout is contractive for each fixed write–read frame. To execute this global computation efficiently, we design a hardware-aware parallel algorithm. It supports exact sequence parallelism across devices. At fixed rank, computation scales linearly with sequence length. With a simple non-hierarchical visual backbone, our Base model reaches 82.7% Top-1 on ImageNet-1K with 78M parameters. It exceeds Vim-B without extra fine-tuning by 0.8 percentage points with approximately 20% fewer parameters. It also surpasses Hydra-ViT-B by 1.1 points. Our Small model reaches 81.0% with 20M parameters. Both results use EMA. After C4 masked-language pretraining, a parameter-matched BERT-style encoder achieves an 81.9 average of GLUE development task means.

1 Introduction

Error-correcting memories have strengthened causal sequence models [32, 17, 12]. These models must retain useful associations in a limited state. Self-attention retains an explicit context for retrieval [26]. Linear attention and state-space models (SSMs) compress that context into a fixed-size memory [16, 9]. Within an associative memory, additive writes can interfere with stored associations. The delta rule corrects this interference: it predicts content from the current memory, then writes the prediction residual [23, 33]. Gated Delta Networks (GDNs) combine this correction with adaptive forgetting [32]. Recent variants introduce channel-wise forgetting and separate erase/write controls [17, 12]. Together, these methods treat memory as an internal model that learns from context. Test-time training (TTT) makes this perspective explicit [24]. MesaNet takes it further by solving for locally optimal regression states [27].

In bidirectional settings, the complete context is available when memory is constructed. All associations can therefore contribute to the same state. This state can use evidence from both sides of any position, without having to serve every intermediate prefix. ViT^3 explores a related opportunity in vision: its full-batch inner updates can outperform sequential mini-batch updates [11]. However, additional inner steps increase cost and can impair stability. A direct equilibrium would let all associations act jointly without an inner iteration budget.

The challenge is to retain the efficiency of compact memory. Vim, Hydra, and LION already offer efficient bidirectional context mixing [34, 14, 1]. For delta-rule memory, each correction also depends on the current state. Removing the causal mask does not remove that dependence. LION's Appendix B.7 derives recurrent and chunkwise bidirectional DeltaNet forms [1]. Its direct full-attention expression, however, requires a sequence-sized matrix inverse. The inversion costs cubic time: doubling the sequence length multiplies that cost by eight. A compact memory thus loses its efficiency advantage if the joint solve grows with the sequence.

We introduce **MemSolve**, which constructs full-context memory through a direct solve in compact memory space. Its design connects joint error correction, controlled readout, and parallel computation.

Global Equilibrium (§2, §3.1). MemSolve combines all associations in one residual equation. An input-dependent soft frame sets the coordinates for writing and reading memory. An accretive core makes the equation uniquely solvable. We couple the frame to the regularizer, reducing the memory equation to a learned core shared across inputs. We solve it directly and read the state back through the same frame. The readout adds a low-rank contextual correction to a content branch. For each realized frame, we prove that the resulting token map is contractive. Its exact spectrum can also be computed in compact space.

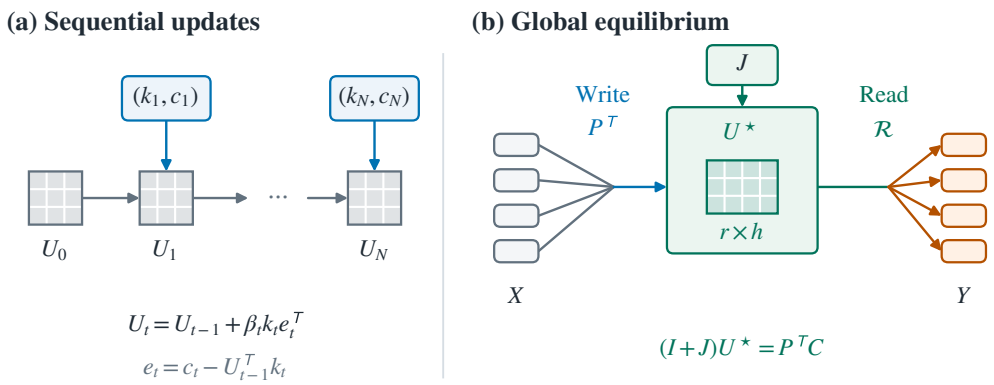


Figure 1: **From sequential memory updates to global equilibrium.** (a) Ordered updates correct one association at a time. (b) MemSolve combines all associations in one shared equilibrium and reads it back to every token. The core J is learned; the frame P and content C depend on the input. Figure 2 details the operator.

Beyond Directional Scans (§3.2). MemSolve is not a reparameterization of a standard bidirectional SSM mixer with local coefficients and a linear merge of the two directions. Such a mixer adds a prefix response to a suffix response. MemSolve instead normalizes its frame using the entire context. Changing the suffix can therefore change how prefix content reaches a token, even when that token is fixed. Theorem 3.3 proves this separation with a three-token counterexample.

Parallel Algorithm (§4). The full context enters the solve through two compact sufficient statistics. We compute these statistics in parallel, solve the compact system, and read out each token independently. The forward pass exchanges only these statistics between sequence shards. This partition preserves the operator exactly. The learned core is shared across inputs and evaluated once per head per forward pass. Selective fusion avoids storing the token-sized frame and reduces GPU memory traffic. The total cost is linear in sequence length at fixed rank.

Architectures and Results (§5). We use the same operator in plain visual backbones and a BERT-style encoder. On ImageNet-1K, MemSolve-ViT-S and MemSolve-ViT-B reach 81.0% and 82.7% Top-1 with 20M and 78M parameters, respectively. Both results use EMA. The Base model exceeds Vim-B without extra fine-tuning by 0.8 percentage points with approximately 20% fewer parameters. The Small model also reaches the same validation accuracy in fewer epochs than an author-released DeiT-S run. The parameter-matched language encoder achieves an 81.9 average of GLUE development task means. Execution measurements show where exact sequence parallelism improves training throughput. Appendix D expands the connections to memory updates, regression layers, and bidirectional sequence operators.

2 MemSolve

MemSolve builds one memory from all positions. We first express this task as joint error correction. We then choose the memory coordinates and regularization so that the equilibrium has a compact, shared system. Finally, we read the memory back to tokens with a controlled gain. All equations describe one head with N valid tokens, rank r, and content width h.

2.1 Correcting all associations together

Let $C \in \mathbb{R}^{N\times h}$ contain the content to be stored, and let $P \in \mathbb{R}^{N\times r}$ assign memory coordinates to each position. A memory $U \in \mathbb{R}^{r\times h}$ predicts content as PU. For row p_i^T of P and row c_i^T of C, the prediction error is $c_i^\mathsf{T} - p_i^\mathsf{T} U$. Writing all errors back gives

$$P^\mathsf{T}(C - PU) = \sum_{i=1}^{N} p_i (c_i^\mathsf{T} - p_i^\mathsf{T} U). \tag{1}$$

Each association is evaluated against the same memory. This is the full-context counterpart of correcting associations one at a time.

To control the joint correction, we add a positive-definite regularizer Γ_X and a learned skew-symmetric coupling Ω:

$$\dot{U} = P^\mathsf{T}(C - PU) - \Gamma_X U - \Omega U. \tag{2}$$

The regularizer damps the memory. The skew term couples its coordinates with zero contribution to memory energy: $\langle U, \Omega U\rangle_F = 0$. MemSolve obtains the memory by solving for the equilibrium of this field. The next step is to construct P and Γ_X so that this solve is well posed and its core can be shared across inputs.

2.2 Constructing a compact equilibrium

A learned projection produces relation features $B \in \mathbb{R}^{N\times r}$ and content C from the input X. We normalize

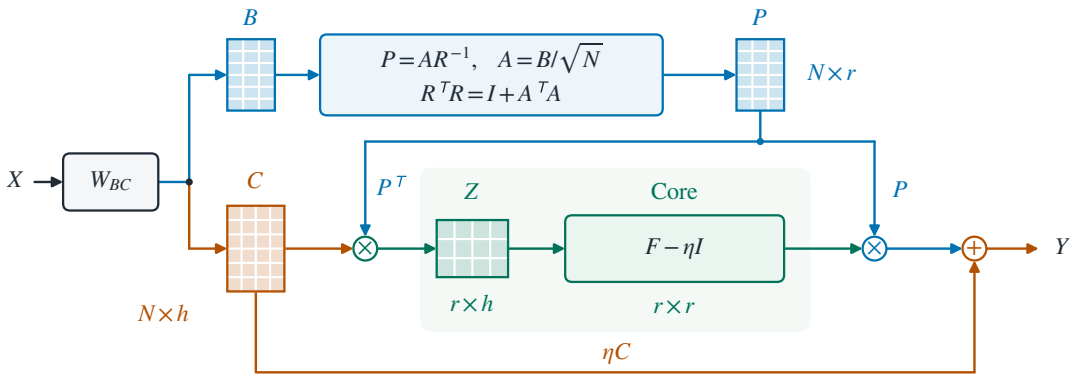


Figure 2: **One MemSolve head.** Relation features form the soft frame P. The head writes $Z = P^\mathsf{T} C$, applies the core correction, and reads through P before adding ηC. Circled multiplication symbols denote matrix products; head concatenation and an output projection follow. This mathematical view makes P explicit; the CUDA path uses compact statistics without storing it.

the relations and define

$$\begin{aligned} A &= B/\sqrt{N}, \quad & R^\mathsf{T} R &= \mathbf{I}_r + A^\mathsf{T} A, \\ P &= AR^{-1}, & Z &= P^\mathsf{T} C. \end{aligned} \tag{3}$$

where R is the upper Cholesky factor with positive diagonal. The normalization makes $A^\mathsf{T} A$ an average relation second moment. The identity term keeps the factorization positive definite and ensures $\|P\|_2 < 1$. We call P a *soft frame*: it provides shared write–read coordinates with bounded gain. Appendix A.1 gives its singular values and geometry.

Each head learns a core shared across samples:

$$J = LL^\mathsf{T} + \Omega, \qquad LL^\mathsf{T} \succ 0, \qquad \Omega^\mathsf{T} = -\Omega. \tag{4}$$

Its symmetric part is positive definite, so J is strictly accretive. We use a lower triangular L with positive diagonal; the unconstrained parameterization is given in Appendix B.1. Choose the regularizer to match the frame:

$$\Gamma_X = \mathbf{I}_r + LL^\mathsf{T} - P^\mathsf{T} P \succ 0. \tag{5}$$

Positivity follows from $P^\mathsf{T} P \prec \mathbf{I}_r$. This choice cancels the frame Gram term in Eq. (2):

$$\dot{U} = Z - (P^\mathsf{T} P + \Gamma_X + \Omega)U = Z - (\mathbf{I}_r + J)U. \tag{6}$$

The equilibrium and its reflected map are therefore

$$\begin{aligned} U^\star &= (\mathbf{I}_r + J)^{-1} Z, \\ F &= 2(\mathbf{I}_r + J)^{-1} - \mathbf{I}_r \\ &= (\mathbf{I}_r + J)^{-1}(\mathbf{I}_r - J). \end{aligned} \tag{7}$$

Here and below, inverse notation denotes a linear solve. The learned J and F are shared across samples. Each input supplies its own P, Z, and $U^\star$. Thus one core factorization serves the batch, while the frame factorization remains input-dependent. No inner iteration count or learning rate is needed.

The cancellation is a design choice: the frame and regularizer jointly define the memory problem. When $\Omega = 0$, the equilibrium minimizes

$$\mathcal{E}_X(U) = \tfrac{1}{2}\,\|PU - C\|_F^2 + \tfrac{1}{2}\,\mathrm{tr}(U^\mathsf{T} \Gamma_X U). \tag{8}$$

With nonzero Ω, it solves a strongly monotone linear equation. The same existence guarantee holds, although the field is no longer the gradient of this scalar objective.

2.3 Reading the memory back to tokens

The equilibrium alone specifies a memory, not a token output. We read it through the same coordinates used for writing. First, form the reflected response $2U^\star - Z = FZ$. This gives a contractive compact map F (Theorem 3.2). Its parameterization covers every real strict contraction in r dimensions, including nonsymmetric maps.

We combine this contextual response with a content response:

$$Y = \eta(\mathbf{I}_N - PP^\mathsf{T})C + PFP^\mathsf{T} C \tag{9}$$

$$= \underbrace{\eta C}_{\text{content}} + \underbrace{P(F - \eta \mathbf{I}_r)P^\mathsf{T} C}_{\text{contextual correction}}. \tag{10}$$

Here $\eta = \tanh(\eta_{\text{raw}}) \in (-1, 1)$ is learned per head. Equivalently, $Y = \eta C + P[2U^\star - (1+\eta)Z]$. The factor two comes from the reflection in Eq. (7). The paired terms in Eq. (10) preserve the gain bound when lifting F to token space. Theorem 3.2 makes this precise. Head outputs are concatenated and passed through a learned output projection.

The default EQUILIBRIUM mode learns F through J. Setting $F = 0$ gives COMPLEMENT, which omits the core parameters:

$$Y_{\text{com}} = \eta(\mathbf{I}_N - PP^\mathsf{T})C = \eta(\mathbf{I}_N + AA^\mathsf{T})^{-1} C. \tag{11}$$

Complement still mixes context through P. Neither PP^T nor $\mathbf{I}_N - PP^\mathsf{T}$ is an orthogonal projector. We initialize

Equilibrium at $J = \mathbf{I}_r$, hence $F = 0$, and use $\eta = 0.9$ in both modes. With matched remaining parameters, the modes start from the same function. Equilibrium then learns the compact map, while both modes learn the frame, content, output projection, and η.

Optional rotary value encoding. Relative position can enter through the content pathway without separating write and read coordinates. Following RoVE [7], we rotate content before aggregation and apply the inverse rotation at readout. For column vectors c_i and rotations $O_i = O(t_i)$ at positions t_i,

$$y_i = \eta c_i + \sum_j D_{ij} O_i^\mathsf{T} O_j c_j, \qquad D = P(F - \eta \mathbf{I}_r)P^\mathsf{T}. \tag{12}$$

At a fixed frequency scale, $O_i^\mathsf{T} O_j = O(t_j - t_i)$ encodes relative offset. This extension adds no learned parameters and preserves the fixed-frame gain bound. Appendix B.7 gives the compact computation and settings.

3 Properties of the Operator

The core equation determines whether memory exists; the readout determines how it affects tokens. We analyze both in exact arithmetic for one head, then distinguish the resulting input dependence from bidirectional scans. Proofs are in Appendix A.

3.1 A unique memory and a bounded readout

Proposition 3.1 (Well-posed memory)**.** *Let $\mu = 1 + \sigma_{\min}(L)^2 > 1$. Then Eq.* (6) *has the unique equilibrium in Eq.* (7)*, and*

$$\left\|(\mathbf{I}_r + J)^{-1}\right\|_2 \le \mu^{-1}, \qquad \left\|(\mathbf{I}_r + J)^{-\mathsf{T}}\right\|_2 \le \mu^{-1}. \tag{13}$$

Every trajectory obeys

$$\|U(t) - U^\star\|_F \le e^{-\mu t} \|U(0) - U^\star\|_F .$$

The same bound controls right-hand-side perturbations in the forward and adjoint solves. It does not depend on the number of tokens.

Theorem 3.2 (Contractive compact and token maps)**.** *The map F in Eq.* (7) *satisfies*

$$\mathbf{I}_r - F^\mathsf{T} F = 4(\mathbf{I}_r + J)^{-\mathsf{T}} L L^\mathsf{T} (\mathbf{I}_r + J)^{-1} \succ 0. \tag{14}$$

For a realized frame P, define

$$K_X = \eta(\mathbf{I}_N - PP^\mathsf{T}) + PFP^\mathsf{T}. \tag{15}$$

Then $\|K_X\|_2 \le \max\{|\eta|, \|F\|_2\} < 1$ and $\operatorname{rank}(K_X - \eta \mathbf{I}_N) \le r$. Moreover, $K_X v = \eta v$ for every $v \in \ker(P^\mathsf{T})$.

This makes the division of work explicit: the content path preserves a scaled response outside the relation span, while the learned core changes interactions within that span. Sharing the write and read frame does not force symmetry: when F is nonsymmetric, PFP^T may be nonsymmetric too. The accretive Cayley parameterization covers every real strict contraction in r dimensions (Appendix A.3); the rank and shared-frame structure still constrain the token operator.

The norm statement holds for a fixed realized frame: it bounds the content map $C \mapsto K_X C$, rather than the derivative through $P(X)$ or the surrounding residual block.

The exact singular values of the token map can also be recovered from a matrix of size at most r. Appendix A.4 gives this compact representation and its proof.

3.2 Interactions across both sides of a token

Global frame normalization lets one side of a token change how the other side is read. A three-token example shows this directly. Set $A = (1, 1, t)^\mathsf{T}$, $C = (s, 0, 0)^\mathsf{T}$, $F = 0$, and $\eta \neq 0$. Equation (11) gives

$$y_2 = -\frac{\eta s}{3 + t^2}. \tag{16}$$

The center token is fixed. Yet changing the suffix variable t changes the coefficient of the prefix content s.

A locally parameterized bidirectional state-space mixer has the form

$$\begin{aligned} h_i^+ &= A^+(x_i) h_{i-1}^+ + b^+(x_i), \\ h_i^- &= A^-(x_i) h_{i+1}^- + b^-(x_i), \\ y_i &= C^+(x_i) h_i^+ + C^-(x_i) h_i^- + d(x_i). \end{aligned} \tag{17}$$

Its boundary states are input-independent. The coefficients may be nonlinear functions of the current token, and the state dimension is unrestricted. Once x_i is fixed, the output separates into $y_i = f_i(x_{1:i}) + g_i(x_{i:N})$. Changing the suffix cannot change the prefix response. This additive structure cannot express Eq. (16) for all s, t.

Theorem 3.3 (Separation from locally parameterized bidirectional SSMs)**.** *For every positive rank r, the MemSolve family contains a map that cannot be represented by Eq.* (17) *for all inputs, irrespective of that mixer's state dimension. The result holds for both Equilibrium and Complement.*

Appendix A.5 proves the result by a mixed finite difference and extends the example to any rank. The theorem concerns individual mixers with local coefficients and linear directional merging. Stacks, nonlinear merges, and globally conditioned coefficients define different model classes.

4 Hardware-Aware Parallel Computation

A compact memory alone does not guarantee efficient execution. Materializing the frame adds token-sized storage and memory traffic. We instead reduce the input to two compact statistics, solve in memory space, and read out token rows in parallel.

4.1 Reduce, solve, and read out

Substituting $P = AR^{-1}$ into Eq. (10) gives

$$\begin{aligned} G &= A^\mathsf{T} A, \qquad H = A^\mathsf{T} C, \\ R^\mathsf{T} R &= \mathbf{I}_r + G, \qquad Z = R^{-\mathsf{T}} H, \\ W &= R^{-1}(F - \eta \mathbf{I}_r) Z, \\ Y &= \eta C + AW. \end{aligned} \tag{18}$$

Only G and H aggregate information across tokens. Once W is available, each output row depends only on its local A, C. Algorithm 1 gives this computation. Equilibrium and Complement use the same steps; Complement takes $F = 0$.

Algorithm 1 MemSolve: compact forward computation (one sample and head)

Input: Relations $B \in \mathbb{R}^{N \times r}$, content $C \in \mathbb{R}^{N \times h}$; shared F, scalar η; $N > r$ valid tokens
Output: Token output $Y \in \mathbb{R}^{N \times h}$

$A \leftarrow B/\sqrt{N}$
$G \leftarrow A^\mathsf{T} A$; $H \leftarrow A^\mathsf{T} C$ ▷ Parallel token reductions
$R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + G)$ ▷ $R^\mathsf{T} R = \mathbf{I}_r + G$
$Z \leftarrow \mathrm{solve}(R^\mathsf{T}, H)$
$W \leftarrow \mathrm{solve}(R, (F - \eta \mathbf{I}_r) Z)$
return $\eta C + AW$ ▷ Parallel token readout

The learned F is evaluated once per head in each forward pass. Each sample then requires $O(Nr^2 + Nrh + r^3 + r^2 h)$ arithmetic and $O(r^2 + rh)$ compact storage per head. Token storage is linear in N. Input and output projections also have linear sequence cost at fixed width. For physical length $N \le r$, an equivalent token-space solve avoids subtraction in the complementary term. Its dimension is at most r. Appendix B.2 gives both length regimes and padding handling.

4.2 Exact sequence parallelism

The two statistics are sums over token rows. Partition those rows across p devices, using the same global valid count N to normalize each shard:

$$\begin{aligned} A_s &= B_s/\sqrt{N}, \\ G = \sum_{s=1}^{p} A_s^\mathsf{T} A_s, &\qquad H = \sum_{s=1}^{p} A_s^\mathsf{T} C_s. \end{aligned} \tag{19}$$

A sum reduction gives each device the centralized G, H. Each device computes the same W and reads out its own rows as $Y_s = \eta C_s + A_s W$. The statistic payload is $O(r^2 + rh)$ per sample and head, independent of sequence length. No token activation gather is needed.

The backward pass likewise reduces a compact readout adjoint before computing local token gradients. Outputs and reduced gradients equal the unsharded computation in real arithmetic; floating-point reduction order can change rounding. Algorithm 5 specifies the reductions and the treatment of shared parameter gradients.

4.3 Selective fusion

The main memory traffic comes from token projections, statistics, and readout. We fuse token operations around their compact intermediates and keep factorizations separate. This avoids storing P and lets the factorizations use specialized solvers. The backward pass reuses compact factors. It also reduces core gradients across the batch without storing one core adjoint per sample.

The GPU implementation uses Triton for tiled token operations, cuSOLVERDx for compact factorizations, and cuBLASLt for projection gradients. Compact solves and adjoints retain FP32 storage. The readout splits sensitive coefficients into a BF16 leading word and a BF16 residual word [13]. Both products accumulate in FP32 before output rounding. This improves coefficient precision while retaining Tensor Core products. Appendix B.6 specifies precision, scheduling, and the surrounding compilation and CUDA Graph execution.

5 Experiments

We evaluate whether the same equilibrium operator supports visual recognition and bidirectional language modeling, and whether its sufficient statistics provide useful sequence parallelism. All MemSolve models below use Equilibrium unless explicitly labeled Complement. The reported experiments do not enable the optional RoVE extension.

5.1 ImageNet classification

Model and protocol. MemSolve-ViT is a non-hierarchical backbone with non-overlapping 16×16 patches and twelve constant-width blocks. Each block adds residual 3×3 depthwise CPE [4], then applies pre-LayerNorm MemSolve and GELU FFN branches with residual connections and stochastic depth (Fig. 3). A final token-wise LayerNorm precedes mean pooling and a linear classifier. We report completed Small and Base runs; Appendix C.1 gives the model configurations.

We train from scratch on ImageNet-1K [22] for 300 epochs at 224×224 with a recipe derived from plain ViT^3. Both models use AdamW, cross-entropy, Mixup/CutMix, RandAugment, and repeated augmentation, with global batch 2,048 and peak learning rate 2×10^{-3}. We evaluate EMA weights with decay 0.9999 and report the checkpoint with the highest validation Top-1. Validation uses bicubic resize to a shorter side of 256 and a 224×224 center crop. Appendix C.1 gives the full recipe.

Results. MemSolve-ViT-S reaches 81.0% with 20M parameters (Table 1). It exceeds DeiT-S, isotropic ConvNeXt-S, Vim-S, and LION-d$^\natural$-S, while remaining below ViG-S, ViT^3-S, and Vim-S with long-sequence fine-tuning. MemSolve-ViT-B reaches 82.7% with 78M parameters. It exceeds Vim-B's standard-training result (81.9%, 98M) by 0.8 percentage points with approximately 20% fewer parameters. It also outperforms Hydra-

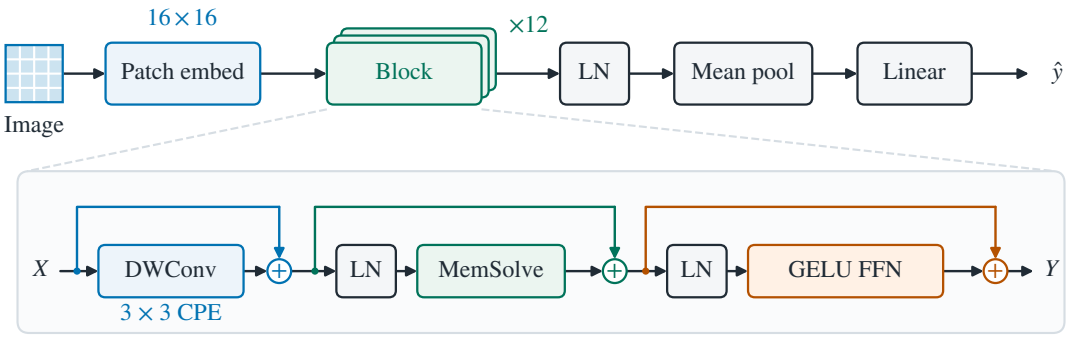


Figure 3: **MemSolve-ViT.** The constant-width backbone stacks twelve blocks and uses mean pooling. The expanded block has three residual branches: depthwise CPE, pre-LayerNorm MemSolve, and pre-LayerNorm GELU FFN. Stochastic depth on the latter two branches is omitted.

Method	Type	#Params	Top-1
ConvNeXt-S (iso.) [19]	ConvNet	22M	79.7
DeiT-S [25]	Transformer	22M	79.8
Vim-S [34]	SSM	26M	80.3
Vim-S† [34]	SSM	26M	81.4
LION-d♮-S [1]	Linear	22M	79.9
ViG-S [18]	Linear	23M	81.7
ViT^3-S [11]	TTT	24M	81.6
MemSolve-ViT-S	Equilibrium	20M	81.0
ConvNeXt-B (iso.) [19]	ConvNet	87M	82.0
ViT-B [14]	Transformer	87M	80.6
DeiT-B [25]	Transformer	86M	81.8
Vim-B [34]	SSM	98M	81.9
Vim-B† [34]	SSM	98M	83.2
Hydra-ViT-B [14]	SSM	91M	81.6
LION-d♮-B [1]	Linear	86M	80.2
ViG-B [18]	Linear	89M	82.6
ViT^3-B [11]	TTT	90M	82.6
MemSolve-ViT-B	Equilibrium	78M	82.7

Table 1: **Non-hierarchical models on ImageNet-1K.** Top-1 is in percent. All inputs are 224 × 224. Baselines follow their published training protocols; MemSolve reports EMA results. †: additional long-sequence fine-tuning; ♮: multiple scans. Appendix C.1 details the reporting conventions.

ViT-B (81.6%, 91M) by 1.1 points. It is close to ViG-B and ViT^3-B, which report 82.6% with 89M and 90M parameters.

Convergence. MemSolve-ViT-S reaches the same validation accuracy in fewer epochs than the author-released DeiT-S run [25] (Fig. 4). Under their respective 300-epoch recipes, MemSolve-ViT-S first reaches 70% Top-1 at epoch 65, compared with epoch 118 for DeiT-S. It reaches 75% at epoch 148 rather than 192. These thresholds require approximately 45% and 23% fewer epochs, respectively. Both curves evaluate ordinary weights.

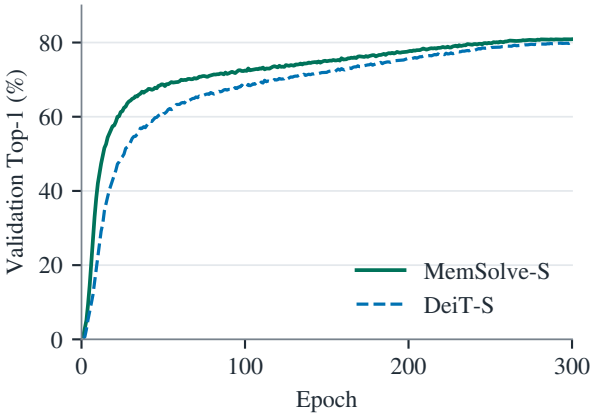


Figure 4: **ImageNet-1K convergence from training logs.** Validation at 224 × 224; every recorded epoch is shown without smoothing. Both models train for 300 epochs and evaluate ordinary weights. Recipes differ, and epochs do not measure wall-clock time. Appendix C.1 gives the log sources.

5.2 Masked language modeling and GLUE

A parameter-matched encoder. We replace BERT's attention mixer with MemSolve in a 14-layer encoder with width 768, twelve heads, rank 32, and a SwiGLU FFN with intermediate width 2,048 per branch. It has 112M parameters including the tied MLM head, close to the 110M BERT-Base scale [6]. The model uses BERT's post-LayerNorm residual structure and learned absolute positions. A centered residual depthwise convolution of width three precedes the relation/content projection in every layer. It starts as the identity and receives only the corrupted MLM input. The completed model does not use rotary encoding.

Pretraining. We follow the LION/M2-derived C4 masked-language setup [21, 1]: sequence length 128, global batch 4,096, 70,000 optimizer steps, and training mask probability 0.30. Local C4 files are streamed and documents are individually truncated; independent docu-

Table 2: **GLUE development results at Base scale.** Published baselines are from LION Table 10, M2-BASE rows; MemSolve reports per-task means. MNLI averages matched/mismatched accuracy, QQP averages accuracy/F1, MRPC uses F1, STS-B uses Spearman, and CoLA uses Matthews correlation; other tasks use accuracy. Avg. is the unweighted mean of the eight task scores. Baseline pretraining, checkpoint selection, and seed aggregation are not matched to our runs.

Model	MNLI	RTE	QQP	QNLI	SST-2	STS-B	MRPC	CoLA	Avg.
BERT	84.6	64.3	90.0	89.8	92.5	86.7	89.6	60.4	82.3
LION-LIT	82.5	63.5	89.7	89.3	91.7	87.2	89.4	49.2	80.3
LION-D	82.9	52.5	89.6	88.4	91.9	86.0	83.9	53.6	78.6
LION-S	83.2	53.5	89.4	88.9	93.0	37.7	77.9	53.2	72.1
MemSolve	83.0	74.2	89.8	88.6	91.6	89.2	91.0	47.5	81.9

ments are not packed into one attention context. AdamW uses peak learning rate 8×10^{-4}, 6% linear warmup, and linear decay to 2% of the peak, following the public code setting. Appendix C.2 states the optimizer conversion and differences from the paper description. On a fixed 10,000-document validation subset with 15% masking, the selected checkpoint reaches 66.4% masked-token accuracy and masked-token NLL 1.6211 over 169,544 targets. The complete run takes 31.80 hours on two A800-80GB GPUs, including scheduled validation and checkpointing.

Downstream evaluation. We fine-tune on eight GLUE tasks using the LION paper's M2-BASE task hyperparameters [28, 1]. MNLI, QQP, QNLI, SST-2, and CoLA start from the MLM checkpoint; RTE, MRPC, and STS-B start from the fine-tuned MNLI encoder. All tasks use length 128 and final-checkpoint development evaluation. We repeat the smaller tasks with three to five seeds while retaining one run for MNLI, QQP, and QNLI.

MemSolve achieves an average of 81.9 (Table 2). This exceeds the reported LION M2-BASE aggregates and is 0.4 points below the reported BERT aggregate. Relative to BERT, RTE has the largest gain and CoLA the largest shortfall. Appendix C.3 reports each task's seed count and standard deviation. The average combines task means from different numbers of fine-tuning runs.

5.3 Sequence-parallel execution

We measure forward and backward for a complete MemSolve layer, including input/output projections and parameter gradients, with batch one, width 768, twelve heads, and rank 32. Table 3 compares one A800-80GB GPU with two-way sequence parallelism on the same GPU type. Synthetic inputs are BF16 and parameters are FP32. CUDA Graph replay excludes compilation and capture; the two-device measurement includes collectives and uses the slower rank's wall time. Optimizer steps are excluded.

At 32,768 tokens, sequence parallelism gives training-computation speedups of $1.66\times$ for Equilibrium and $1.74\times$ for Complement. At 128 tokens, communication and replicated compact work outweigh the saved token work. The partition becomes useful as token work grows relative to communication. These measurements test execution scaling; they do not measure task accuracy at longer lengths.

Table 3: **Exact sequence-parallel training computation.** Times are milliseconds for forward plus backward; speedup is one-device time divided by two-device time. Each time is the median of three blocks of 50 graph replays after warmup.

Mode	Tokens	1 GPU	2 GPUs	Speedup
Equilibrium	128	0.322	0.489	$0.66\times$
	8,192	0.707	0.647	$1.09\times$
	32,768	2.221	1.341	$1.66\times$
Complement	128	0.132	0.322	$0.41\times$
	8,192	0.692	0.586	$1.18\times$
	32,768	2.090	1.202	$1.74\times$

6 Discussion and Limitations

MemSolve concentrates contextual changes in a learned relation span. Rank controls the number of modified directions and the cost of the compact systems. Outside that span, the operator retains a scalar content response. Sharing write and read coordinates supports the contraction guarantee, but it also ties where information is stored to where it is retrieved. These choices define the operator's capacity and computational tradeoff.

The experiments show that this construction learns useful representations in vision and language. Length generalization remains unresolved. The language experiments train and fine-tune at 128 tokens with learned absolute positions. The sequence-parallel benchmark establishes execution scaling, while task accuracy beyond the training length remains to be measured. ImageNet and MLM pretraining each use one seed per configuration. Repeated GLUE runs measure fine-tuning variation from a shared pretrained model.

7 Conclusion

MemSolve constructs full-context associative memory by solving a joint error-correction equilibrium. Its frame and regularizer reduce the memory equation to a shared compact core. The reflected readout gives a contractive token map for each fixed frame. Global normalization also couples evidence across both sides of a token, beyond the

additive responses of locally parameterized bidirectional scans. Two compact statistics make this construction linear in sequence length at fixed rank and support exact sequence parallelism. The visual and language results demonstrate its use as a general token mixer.

References

[1] Arshia Afzal, Elias Abad Rocamora, Leyla Naz Candogan, Pol Puigdemont, Francesco Tonin, Yongtao Wu, Mahsa Shoaran, and Volkan Cevher. Linear attention for efficient bidirectional sequence modeling. In *Advances in Neural Information Processing Systems*, volume 38, 2025.

[2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[3] Heinz H. Bauschke and Patrick L. Combettes. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. CMS Books in Mathematics. Springer, Cham, 2 edition, 2017.

[4] Xiangxiang Chu, Zhi Tian, Bo Zhang, Xinlong Wang, and Chunhua Shen. Conditional positional encodings for vision transformers. In *International Conference on Learning Representations*, 2023.

[5] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

[6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, volume 1, pages 4171–4186, 2019.

[7] Alejandro García-Castellanos, Maurice Weiler, and Erik J. Bekkers. RoVE: Rotary value embeddings attention for relative position-dependent value pathways. *arXiv preprint arXiv:2606.11275*, 2026.

[8] Marta Garnelo and Wojciech Marian Czarnecki. Exploring the space of key-value-query models with intention. *arXiv preprint arXiv:2305.10203*, 2023.

[9] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling*, 2024.

[10] Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

[11] Dongchen Han, Yining Li, Tianyu Li, Zixuan Cao, Ziming Wang, Jun Song, Yu Cheng, Bo Zheng, and Gao Huang. ViT^3: Unlocking test-time training in vision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 51–61, 2026.

[12] Ali Hatamizadeh, Yejin Choi, and Jan Kautz. Gated DeltaNet-2: Decoupling erase and write in linear attention. *arXiv preprint arXiv:2605.22791*, 2026.

[13] Greg Henry, Ping Tak Peter Tang, and Alexander Heinecke. Leveraging the bfloat16 artificial intelligence datatype for higher-precision computations. *arXiv preprint arXiv:1904.06376*, 2019.

[14] Sukjun Hwang, Aakash Lahoti, Ratish Puduppully, Tri Dao, and Albert Gu. Hydra: Bidirectional state space models through generalized matrix mixers. In *Advances in Neural Information Processing Systems*, volume 37, 2024.

[15] Timothy Kassis, Vinayak Agarwal, Yuhuan He, Darshil Patel, and Aubrey M. Brueckner. Scientific agent skills: A library of procedural knowledge for research agents. *arXiv preprint arXiv:2609.00065*, 2026.

[16] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020.

[17] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, et al. Kimi linear: An expressive, efficient attention architecture. *arXiv preprint arXiv:2510.26692*, 2025.

[18] Bencheng Liao, Xinggang Wang, Lianghui Zhu, Qian Zhang, and Chang Huang. ViG: Linear-complexity visual sequence learning with gated linear attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 5182–5190, 2025.

[19] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022.

[20] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. YaRN: Efficient context window extension of large language models. In *International Conference on Learning Representations*, 2024.

[21] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[22] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115:211–252, 2015.

[23] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, 2021.

[24] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, Tatsunori Hashimoto, and Carlos Guestrin. Learning to Learn at test time: RNNs with expressive hidden states. *arXiv preprint arXiv:2407.04620*, 2025.

[25] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers and distillation through attention. In *International Conference on Machine Learning*, pages 10347–10357, 2021.

[26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[27] Johannes von Oswald, Nino Scherrer, Seijin Kobayashi, Luca Versari, Songlin Yang, Sarthak Mittal, Maximilian Schlegel, Kaitlin Maile, Yanick Schimpf, Oliver Sieberling, Alexander Meulemans, Rif A. Saurous, Guillaume Lajoie, Charlotte Frenkel, Razvan Pascanu, Blaise Agüera y Arcas, and João Sacramento. MesaNet: Sequence modeling by locally optimal test-time training. In *International Conference on Learning Representations*, 2026.

[28] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019.

[29] Ke Alexander Wang, Jiaxin Shi, and Emily B. Fox. Test-time regression: A unifying framework for designing sequence models with associative memory. *arXiv preprint arXiv:2501.12352*, 2025.

[30] Ezra Winston and J. Zico Kolter. Monotone operator equilibrium networks. In *Advances in Neural Information Processing Systems*, 2020.

[31] Haocheng Xi, Yiming Xie, Hexu Zhao, Yiwen Zhang, Michael Liu, Thomas Creavin, Kurt Keutzer, Xiuyu Li, Zhaoyang Lv, Chenfeng Xu, and Haiwen Feng. Video DeltaNet: A video-native hybrid attention for livestream video generation. *arXiv preprint arXiv:2609.20744*, 2026.

[32] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations*, 2025.

[33] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. *arXiv preprint arXiv:2406.06484*, 2025.

[34] Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *International Conference on Machine Learning*, pages 62429–62442, 2024.

A Proofs and Structural Details

We first establish the frame geometry. It gives the positive regularizer used in the memory equation. We then prove existence, contraction, and the compact spectrum, followed by the bidirectional-scan separation.

A.1 Soft-frame geometry

Since $R^\mathsf{T} R = \mathbf{I} + A^\mathsf{T} A$,

$$PP^\mathsf{T} = A(\mathbf{I} + A^\mathsf{T} A)^{-1} A^\mathsf{T}. \tag{20}$$

An SVD of A gives

$$\sigma_i(P) = \frac{\sigma_i(A)}{\sqrt{1+\sigma_i(A)^2}} < 1. \tag{21}$$

In particular, $P^\mathsf{T} P \prec \mathbf{I}_r$ and $PP^\mathsf{T} \prec \mathbf{I}_N$, including rank-deficient cases. The Woodbury identity yields

$$\mathbf{I}_N - PP^\mathsf{T} = (\mathbf{I}_N + AA^\mathsf{T})^{-1} \succ 0, \tag{22}$$

which proves Eq. (11) and positivity of Γ_X. The reduced QR factorization of $[A^\mathsf{T}, \mathbf{I}_r]^\mathsf{T}$ with a positive diagonal in its triangular factor has top block P. Thus the QR reference and the compact Cholesky construction agree in exact arithmetic.

The $\sqrt{N}$ normalization controls the scale of the relation statistic. If $N^{-1}\sum_i \|b_i\|_2^2 \le c$, then $\|A\|_2^2 \le \|A\|_F^2 \le c$ and $\kappa_2(\mathbf{I} + A^\mathsf{T} A) \le 1 + c$, independently of N. Under this feature-energy bound, the condition number does not grow with sequence length. Accuracy at unseen lengths is a separate empirical question.

Without a positional module, the operator is permutation equivariant: permuting the rows of A, C leaves G, H unchanged and permutes the rows of Y. The visual and language backbones therefore supply position information outside the core operator.

A.2 Memory existence, sensitivity, and convergence

Write $M = \mathbf{I}_r + J$. For every real v,

$$v^\mathsf{T} M v = v^\mathsf{T}(\mathbf{I}_r + LL^\mathsf{T})v \ge \mu \|v\|_2^2, \qquad \mu = 1 + \sigma_{\min}(L)^2. \tag{23}$$

Cauchy–Schwarz gives $\|Mv\|_2 \ge \mu \|v\|_2$, so M is invertible and $\left\|M^{-1}\right\|_2 \le 1/\mu$. The same argument applies to M^T. For $E(t) = U(t) - U^\star$, $\dot{E} = -ME$ and

$$\frac{d}{dt}\|E\|_F^2 = -2\,\mathrm{tr}(E^\mathsf{T}(\mathbf{I}_r + LL^\mathsf{T})E) \le -2\mu \|E\|_F^2. \tag{24}$$

Integration proves Proposition 3.1. For $\Omega = 0$, differentiating Eq. (8) gives $\nabla_U \mathcal{E}_X = MU - Z$ and Hessian $M \succ 0$ for each content column. When $\Omega \neq 0$, the nonsymmetric field need not be a scalar gradient, although the same convergence argument holds.

A.3 Cayley contraction and coverage

As $M = \mathbf{I} + J$ commutes with $\mathbf{I} - J$,

$$\mathbf{I} - F^\mathsf{T} F = M^{-\mathsf{T}}\left[M^\mathsf{T} M - (\mathbf{I}-J)^\mathsf{T}(\mathbf{I}-J)\right] M^{-1} \tag{25}$$

$$= 2M^{-\mathsf{T}}(J + J^\mathsf{T})M^{-1} = 4M^{-\mathsf{T}} LL^\mathsf{T} M^{-1} \succ 0. \tag{26}$$

Conversely, let F_0 be any real matrix with $\|F_0\|_2 < 1$. Then $\mathbf{I} + F_0$ is invertible and

$$J_0 = (\mathbf{I} - F_0)(\mathbf{I} + F_0)^{-1}, \qquad \mathrm{sym}(J_0) = (\mathbf{I} + F_0)^{-\mathsf{T}}(\mathbf{I} - F_0^\mathsf{T} F_0)(\mathbf{I} + F_0)^{-1} \succ 0. \tag{27}$$

Factor $\mathrm{sym}(J_0) = LL^\mathsf{T}$ and set $\Omega = (J_0 - J_0^\mathsf{T})/2$. The triangular/softplus parameterization can represent every such L and Ω. This is the classical accretive–Cayley correspondence [3]. It establishes coverage of all real strict contractions in compact space; the token map retains its shared-frame and rank constraints.

A.4 Token contraction and the exact compact spectrum

Let $D = (\mathbf{I}_N - PP^\mathsf{T})^{1/2}$ and $V_0 = [D^\mathsf{T}, P]^\mathsf{T} \in \mathbb{R}^{(N+r)\times N}$, so $V_0^\mathsf{T} V_0 = \mathbf{I}_N$. Then

$$K_X = V_0^\mathsf{T} \operatorname{diag}(\eta \mathbf{I}_N, F) V_0, \tag{28}$$

which proves the norm bound in Theorem 3.2. The rank and nullspace statements follow directly from $K_X - \eta \mathbf{I}_N = P(F - \eta \mathbf{I}_r)P^\mathsf{T}$.

Proposition A.1 (Exact analysis in compact space)**.** *Let $P = QSV^\mathsf{T}$ be a thin singular value decomposition with $k = \operatorname{rank}(P)$, $Q^\mathsf{T}Q = V^\mathsf{T}V = \mathbf{I}_k$, and positive diagonal S. The token map is orthogonally similar to*

$$\left[\eta \mathbf{I}_k + SV^\mathsf{T}(F - \eta \mathbf{I}_r)VS\right] \oplus \eta \mathbf{I}_{N-k}. \tag{29}$$

Its singular values, including its exact spectral norm, can therefore be obtained from a matrix of size at most r, together with $N - k$ copies of $|\eta|$.

To prove the proposition, extend Q to an orthogonal basis $[Q, Q_\perp]$. In this basis the low-rank correction acts only on the first k coordinates, giving Eq. (29). If $k = 0$, the result is simply $\eta \mathbf{I}_N$; if $k = N$, the complementary block is absent. One may compute V and S from the positive eigenspace of $P^\mathsf{T}P$, so no $N \times N$ matrix or token-space SVD is required.

A.5 Proof of the bidirectional SSM separation

Fix the center token x_i in Eq. (17). Recursion with input-independent boundary states gives $y_i = f_i(x_{1:i}) + g_i(x_{i:N})$. For any prefix perturbation s and suffix perturbation t that leave x_i fixed, its mixed finite difference is

$$\Delta_{s,t} y_i = y_i(s_1, t_1) - y_i(s_1, t_0) - y_i(s_0, t_1) + y_i(s_0, t_0) = 0. \tag{30}$$

This conclusion holds for arbitrary coefficient functions and state size.

For MemSolve, take three tokens whose two coordinates independently control relation and content: $x_1 = (1, s)$, $x_2 = (1, 0)$, and $x_3 = (t, 0)$. A fixed relation projection with scale $\sqrt{3}$ gives $A = (1, 1, t)^\mathsf{T}$ after the normalization in Eq. (3); a content projection gives $C = (s, 0, 0)^\mathsf{T}$. With $F = 0$,

$$PP^\mathsf{T} = \frac{AA^\mathsf{T}}{1 + A^\mathsf{T}A}, \qquad y_2 = -\frac{\eta s}{3 + t^2}. \tag{31}$$

At $(s_0, s_1) = (0, 1)$ and $(t_0, t_1) = (0, 1)$,

$$\Delta_{s,t} y_2 = -\frac{\eta}{4} + \frac{\eta}{3} = \frac{\eta}{12} \neq 0. \tag{32}$$

Therefore no mixer of Eq. (17) represents this MemSolve map for all inputs. For any larger rank, append zero relation columns; the first Cholesky coordinate and the output above remain unchanged. Complement sets $F = 0$ by definition, and Equilibrium realizes $F = 0$ at $J = \mathbf{I}_r$. This proves Theorem 3.3.

The distinction concerns how inputs determine the map, not the rank of a matrix after fixing the input. A realized MemSolve matrix is low rank plus a diagonal and can admit other algebraic factorizations. Such a factorization does not supply the local coefficient dependence required by Eq. (17).

B Algorithms and Numerical Details

This appendix gives the core parameterization, the masked forward pass, and the analytic backward. It then specifies sequence-parallel reductions and floating-point execution. The pseudocode describes exact arithmetic; Section B.6 explains how the GPU kernels evaluate it.

B.1 Core parameters and initialization

Each head stores an unconstrained matrix $\Theta \in \mathbb{R}^{r\times r}$. Its lower and upper entries define the symmetric and skew parts of J:

$$L = \operatorname{tril}(\Theta, -1) + \operatorname{diag}(\operatorname{softplus}(\operatorname{diag}\Theta + \log(e - 1))), \tag{33}$$

$$\Omega = \operatorname{triu}(\Theta, 1) - \operatorname{triu}(\Theta, 1)^\mathsf{T}, \qquad J = LL^\mathsf{T} + \Omega. \tag{34}$$

The diagonal of L is positive, so $LL^\mathsf{T} \succ 0$. At $\Theta = 0$, $L = \mathbf{I}_r$, $J = \mathbf{I}_r$, and $F = 0$. This initialization preserves the core's training signal: $\delta F = -\frac{1}{2}\delta J$ at $J = \mathbf{I}_r$. The initial learned correction is zero, but its derivative need not be.

The per-head scalar is parameterized by $\eta = \tanh(\eta_{\text{raw}})$ and initialized to 0.9. Floating-point execution uses

$$\eta_{\text{fp}} = (1 - \epsilon_{\text{mach}}) \tanh(\eta_{\text{raw}}), \qquad \frac{d\eta_{\text{fp}}}{d\eta_{\text{raw}}} = (1 - \epsilon_{\text{mach}}) \frac{4q}{(1+q)^2}, \quad q = e^{-2|\eta_{\text{raw}}|}. \tag{35}$$

The scale keeps the value inside $(-1, 1)$. Native $\tanh$ preserves small values near zero; the exponential derivative retains representable tail gradients after $\tanh$ rounds to an endpoint.

B.2 Forward computation

Algorithm 2 expands Algorithm 1 to masked batches and short sequences. The input and output projections are ordinary affine layers. Head outputs are concatenated, projected, and masked again so that output bias cannot populate padding.

Physical length is N; sample b has n_b valid tokens. Masks multiply rows and are fixed under differentiation. An all-masked sample uses a divisor of one and produces zero. We use $\text{solve}(M, V)$ for the solution of $MU = V$ and $\text{chol}_u(S)$ for the upper factor R satisfying $R^\mathsf{T} R = S$. Thus

$$\text{cholsolve}(R, V) = \text{solve}\big(R, \text{solve}(R^\mathsf{T}, V)\big). \tag{36}$$

A general core system uses LU factorization. For $N \le r$, the complementary term uses the equivalent solve $\eta\,\text{solve}(\mathbf{I}_N + AA^\mathsf{T}, C)$. This avoids subtracting nearly equal terms in $\eta(C - PP^\mathsf{T}C)$. Equilibrium adds its learned correction to that result. The largest factorization has dimension at most r in either regime.

Algorithm 2 MemSolve forward without materializing P (one head)

Input: $\{B_b \in \mathbb{R}^{N\times r}, C_b \in \mathbb{R}^{N\times h}, m_b \in \{0,1\}^N\}_{b=1}^s$; mode, η; $\Theta \in \mathbb{R}^{r\times r}$ for Equilibrium
Output: Head outputs $\{Y_b \in \mathbb{R}^{N\times h}\}_{b=1}^s$

if mode is Equilibrium **then**
  Construct L, Ω, J from Θ using Eq. (34)
  $M \leftarrow \mathbf{I}_r + J$; factor M once
  $F \leftarrow \text{solve}(M, \mathbf{I}_r - J)$ ▷ Shared across this batch
else
  $F \leftarrow 0$ ▷ Complement
end if
for each sample b in parallel **do**
  $n_b \leftarrow \sum_i m_{b,i}$; $A \leftarrow (m_b \odot B_b)/\sqrt{\max(n_b, 1)}$; $C \leftarrow m_b \odot C_b$
  if $N \le r$ **then** ▷ Physical length determines the branch
    $R_N \leftarrow \text{chol}_u(\mathbf{I}_N + AA^\mathsf{T})$; $Y_b \leftarrow \eta\ \text{cholsolve}(R_N, C)$
    if mode is Equilibrium **then**
      $R \leftarrow \text{chol}_u(\mathbf{I}_r + A^\mathsf{T}A)$; $Z \leftarrow \text{solve}(R^\mathsf{T}, A^\mathsf{T}C)$
      $Y_b \leftarrow Y_b + A\,\text{solve}(R, FZ)$
    end if
  else
    $G \leftarrow A^\mathsf{T}A$; $H \leftarrow A^\mathsf{T}C$; $R \leftarrow \text{chol}_u(\mathbf{I}_r + G)$
    $Z \leftarrow \text{solve}(R^\mathsf{T}, H)$; $W \leftarrow \text{solve}(R, (F - \eta\mathbf{I}_r)Z)$
    $Y_b \leftarrow \eta C + AW$
  end if
end for
return $\{Y_b\}_{b=1}^s$

For $N > r$, the backward below uses A, C, R, W, Z and the shared F. Implementations can save these values or reconstruct them from projections and compact intermediates. Equilibrium also reuses L and the LU factors of $\mathbf{I}_r + J$. No token-sized frame is needed. For $N \le r$, automatic differentiation handles the short-sequence expression, with an analytic adjoint for the shared core solve.

B.3 A common compact backward

Algorithm 3 differentiates Eq. (18) for one sample and head. It applies to both modes: set $F = 0$ for Complement and discard the adjoint of this fixed matrix. A bar denotes a loss adjoint under the Frobenius inner product, and $\text{sym}(T) = (T + T^\mathsf{T})/2$. The algorithm includes both the direct content term and the correction. Their contributions to C and η are added before returning. Projection gradients follow the usual affine-layer rules.

For an upper Cholesky factor R, define

$$\Phi_u(T) = \mathrm{triu}(T, 1) + \tfrac{1}{2}\,\mathrm{diag}(\mathrm{diag}\,T), \qquad \mathcal{C}(R, E_R) = R^{-1}\,\mathrm{sym}\big(\Phi_u(E_R R^{\mathsf{T}})\big)\,R^{-\mathsf{T}}. \tag{37}$$

Here $\mathcal{C}$ is the adjoint of $R = \mathrm{chol}_u(S)$ with respect to the symmetric matrix S; the inverse notation again means two triangular solves. To obtain it, differentiate $S = R^{\mathsf{T}}R$ and observe that $dR = \Phi_u(R^{-\mathsf{T}}(dS)R^{-1})R$. This also fixes the half-diagonal convention in Eq. (37).

Algorithm 3 Rank-space backward ($N > r$, one sample and head)

Input: $E = \overline{Y} \in \mathbb{R}^{N\times h}$; forward values A, C, R, W, Z, F, η; fixed mask m and valid count n
Output: $\overline{B}, \overline{C}_{\mathrm{in}}$; contributions $\overline{\eta}, \overline{F}$

$E_W \leftarrow A^{\mathsf{T}}E$; $\overline{A} \leftarrow EW^{\mathsf{T}}$; $\overline{C} \leftarrow \eta E$
$\overline{\eta} \leftarrow \langle E, C\rangle_F$
$E_T \leftarrow \mathrm{solve}(R^{\mathsf{T}}, E_W)$
$\overline{F} \leftarrow E_T Z^{\mathsf{T}}$; $\overline{\eta} \leftarrow \overline{\eta} - \langle E_T, Z\rangle_F$
$\overline{Z} \leftarrow (F - \eta\mathbf{I}_r)^{\mathsf{T}}E_T$; $\overline{H} \leftarrow \mathrm{solve}(R, \overline{Z})$
$\overline{R} \leftarrow -E_T W^{\mathsf{T}} - Z\overline{H}^{\mathsf{T}}$
$\overline{G} \leftarrow \mathcal{C}(R, \overline{R})$ ▷ Eq. (37)
$\overline{A} \leftarrow \overline{A} + C\overline{H}^{\mathsf{T}} + A(\overline{G} + \overline{G}^{\mathsf{T}})$
$\overline{C} \leftarrow \overline{C} + A\overline{H}$
$\overline{B} \leftarrow m \odot \overline{A}/\sqrt{\max(n, 1)}$
$\overline{C}_{\mathrm{in}} \leftarrow m \odot \overline{C}$
return $\overline{B}, \overline{C}_{\mathrm{in}}, \overline{\eta}, \overline{F}$

Reusing the inverse triangular factor. The GPU path computes $T = R^{-\mathsf{T}}$ once and reuses it in forward and backward products. The compact derivatives above can be evaluated directly in terms of T. Let $M_F = F - \eta\mathbf{I}_r$, $E_T = TE_W$, and $V = E_T Z^{\mathsf{T}}$. Then

$$\overline{H} = T^{\mathsf{T}} M_F^{\mathsf{T}} E_T, \tag{38}$$

$$\overline{G} = T^{\mathsf{T}}\,\mathrm{sym}_\ell(-M_F V^{\mathsf{T}} - M_F^{\mathsf{T}} V)T, \tag{39}$$

where $\mathrm{sym}_\ell(K) = [\mathrm{tril}(K) + \mathrm{tril}(K, -1)^{\mathsf{T}}]/2$. The core and scalar contributions reuse V: $\overline{F} = V$ and $\overline{\eta} = \langle E, C\rangle_F - \mathrm{tr}(V)$. This form shares the same compact adjoint between Equilibrium and Complement.

B.4 Shared-core and scalar gradients

Write $M = \mathbf{I}_r + J$. For $F = M^{-1}(\mathbf{I} - J)$, differentiation gives

$$dF = -M^{-1}(dJ)(\mathbf{I} + F). \tag{40}$$

Since the core is shared, its adjoints are summed over samples before the transposed solve in Algorithm 4. The reduction is a sum: any loss averaging is already contained in $\overline{Y}$. The fast path accumulates this reduction in tiles rather than storing one $r \times r$ core adjoint per sample.

Algorithm 4 Shared-core backward (once per head, Equilibrium only)

Input: $E_F = \sum_b \overline{F}_b$; Θ, L, F and the saved factorization of $M = \mathbf{I}_r + J$
Output: Core-parameter gradient $\overline{\Theta} \in \mathbb{R}^{r\times r}$

$T \leftarrow \mathrm{solve}(M^{\mathsf{T}}, E_F)$ ▷ Reuse the forward LU factors
$\overline{J} \leftarrow -T(\mathbf{I}_r + F)^{\mathsf{T}}$
$\overline{L} \leftarrow (\overline{J} + \overline{J}^{\mathsf{T}})L$
$d \leftarrow \mathrm{diag}(\overline{L}) \odot \mathrm{sigmoid}(\mathrm{diag}\,\Theta + \log(e - 1))$
$\overline{\Theta} \leftarrow \mathrm{tril}(\overline{L}, -1) + \mathrm{diag}(d) + \mathrm{triu}(\overline{J} - \overline{J}^{\mathsf{T}}, 1)$
return $\overline{\Theta}$

For either mode, sum the scalar adjoints over samples before applying $\overline{\eta_{\mathrm{raw}}} = (d\eta/d\eta_{\mathrm{raw}})\sum_b \overline{\eta}_b$. For the mathematical parameterization, $d\eta/d\eta_{\mathrm{raw}} = 1 - \tanh^2(\eta_{\mathrm{raw}})$. The fast path provides first-order training gradients.

B.5 Sequence-parallel reductions

Algorithm 5 expresses the operator for a sum of local losses. A global mean is obtained by normalizing every local loss with the same global denominator. VJP_W denotes the adjoint of $(G, H, F, \eta) \mapsto W$ in Eq. (18). It is the compact part of Algorithm 3 and excludes the direct term ηC. A sum reduction of E_W gives every device the same compact adjoints. Shared F, η gradients are therefore already global and must be counted once. The implementation divides their replicated contributions by the number of devices before the final parameter-gradient sum.

Algorithm 5 Exact sequence-parallel MemSolve (one sample and head)

Input: Local rows B_s, C_s, mask m_s; replicated F, η; p devices; global physical length $N > r$
Output: Local output Y_s, local input gradients, and global parameter gradients

$n \leftarrow \mathrm{AllReduceSum}(\sum_i m_{s,i})$
$A_s \leftarrow (m_s \odot B_s)/\sqrt{\max(n,1)}$; $C_s \leftarrow m_s \odot C_s$
$(G, H) \leftarrow \mathrm{AllReduceSum}(A_s^{\mathsf{T}} A_s, A_s^{\mathsf{T}} C_s)$
$R \leftarrow \mathrm{chol}_u(\mathbf{I}_r + G)$; $Z \leftarrow \mathrm{solve}(R^{\mathsf{T}}, H)$
$W \leftarrow \mathrm{solve}(R, (F - \eta \mathbf{I}_r)Z)$; $Y_s \leftarrow \eta C_s + A_s W$
Backward: receive local output adjoint E_s
$E_W \leftarrow \mathrm{AllReduceSum}(A_s^{\mathsf{T}} E_s)$
$(\overline{G}, \overline{H}, \overline{F}, \overline{\eta}_W) \leftarrow \mathrm{VJP}_W(G, H, F, \eta; E_W)$
$\overline{A}_s \leftarrow E_s W^{\mathsf{T}} + C_s \overline{H}^{\mathsf{T}} + A_s(\overline{G} + \overline{G}^{\mathsf{T}})$
$\overline{C}_s \leftarrow \eta E_s + A_s \overline{H}$
$\overline{\eta} \leftarrow \mathrm{AllReduceSum}(\langle E_s, C_s \rangle_F) + \overline{\eta}_W$
$\overline{B}_s \leftarrow m_s \odot \overline{A}_s/\sqrt{\max(n,1)}$; $\overline{C}_{\mathrm{in},s} \leftarrow m_s \odot \overline{C}_s$
Backpropagate local projection gradients and sum them across devices
Apply the shared-core adjoint to $\overline{F}$ once; retain one copy of $\overline{\eta}$

The proof is the additivity of G, H in the forward pass and the chain rule in the backward pass. No recurrence connects neighboring sequence shards. In a backbone with a centered convolution, that local operation requires a separate halo exchange; it does not change the memory reduction. The implemented sequence-parallel path uses the compact formulation and requires global physical length greater than the operator rank.

B.6 Precision and scheduling

The fast path supports ranks 16, 32, 48, and 64. Token projections and statistics use BF16 multiplicands and FP32 accumulation. Gram matrices, compact states, factors, and adjoints retain FP32 storage. Batched compact products use Triton's three-term TF32 decomposition; small shared-core products and solves retain IEEE FP32 arithmetic. Readout kernels split sensitive FP32 coefficients into BF16 leading and residual words before accumulating both products in FP32. The extra product recovers coefficient precision without using FP32 operands for token-sized products. With very sparse masks and strong relations, cancellation can still amplify relative error in the compact complementary readout.

Measured implementation scope. The completed training runs used NVIDIA A800-SXM4-80GB GPUs, PyTorch 2.14.0 with CUDA 13.2, and timm 1.0.29. Compilation fuses surrounding operations; CUDA Graph replay reduces launch overhead. These execution choices do not alter Eqs. (3)–(10). Table 3 reports the recorded sequence-parallel implementation on this hardware. It compares partitioning strategies for MemSolve rather than matched throughput against other mixers.

B.7 Optional rotary value encoding

Let $\mathcal{O}$ rotate each content row: $(\mathcal{O}C)_i^{\mathsf{T}} = O_i c_i$. The rotations cover all h channels in adjacent pairs, so h is even. The compact forward changes only the content statistic and final readout:

$$\begin{aligned} &\widetilde{C} = \mathcal{O}C, \qquad && H = A^{\mathsf{T}}\widetilde{C}, \qquad && Z = R^{-\mathsf{T}}H, \\ &W = R^{-1}(F - \eta \mathbf{I}_r)Z, \qquad && Y = \eta C + \mathcal{O}^{-1}(AW). \end{aligned} \tag{41}$$

The Gram matrix and core solve are unchanged. Both modes use this form, with $F = 0$ for Complement. The inverse rotation covers the full correction, including its $-\eta \mathbf{I}_r$ term, and precedes the output projection. Backward applies the transposed rotations around the same compact adjoints. The rotations cost $O(Nh)$ and fuse into statistics and readout. Sequence shards use global positions and the same frequency scale.

For a fixed frame, write $\mathcal{K}(C) = K_X C$ with K_X from Eq. (15). Equation (41) is $\mathcal{O}^{-1}\mathcal{K}\mathcal{O}$. Since $\mathcal{O}$ is orthogonal under the Frobenius inner product, this conjugation preserves singular values and the gain bound in Theorem 3.2.

We follow RoVE's value rotations and upstream frequency interpolation [7, 20]. Frequencies are fixed at $\omega_k = 10000^{-2k/h}$ for $k = 0, \ldots, h/2 - 1$. The optional language configuration uses a 128-token training window. Beyond it, frequency-only YaRN retains high frequencies and smoothly scales low frequencies, with $\beta_{\text{fast}} = 32$ and $\beta_{\text{slow}} = 1$. Forward and inverse rotations use the same frequency table and unit amplitude. This implemented extension has not yet been evaluated in the full pretraining and downstream experiments reported here.

C Experimental Protocols and Results

C.1 Visual models and optimization

Table 4 lists the Tiny, Small, and Base configurations.

Table 4: **MemSolve-ViT configurations.** Parameters include the 1,000-class head and the Equilibrium core and are rounded to whole millions. All models have depth 12 and patch size 16; DropPath increases linearly to the listed maximum.

Model	Width	Heads	Rank/head	FFN width	DropPath	Params (M)
Tiny	192	6	16	768	0.0	5
Small	384	6	32	1584	0.1	20
Base	768	12	48	3072	0.4	78

Small uses FFN ratio $1584/384 = 4.125$; Tiny and Base use ratio 4. LayerNorm uses $\epsilon = 10^{-6}$. CPE is a residual depthwise 3×3 convolution with padding one, applied at every block. The visual encoder has neither a class token nor an absolute position table. All model parameters remain FP32 under BF16 autocast.

All completed runs use fused AdamW, weight decay 0.05, gradient clipping at norm 5, cross-entropy with label smoothing 0.1, Mixup 0.8, CutMix 1.0, RandAugment, and random erasing 0.25. The learning rate uses 20 epochs of linear warmup and update-based cosine decay. Validation uses bicubic resize to a shorter side of 256 followed by a 224×224 center crop. No rotary transform or LayerScale is used.

Both Small and Base use three-repeat augmentation, global batch 2,048, and peak learning rate 2×10^{-3}, with 512 images per GPU and two accumulation steps on two A800 GPUs. Repeated augmentation retains a 1.28M-presentation epoch budget. Mixup/CutMix operates on each complete physical microbatch. We evaluate EMA weights and select the checkpoint with the highest validation Top-1.

AdamW uses $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$. RandAugment is `rand-m9-mstd0.5-inc1`, color jitter is 0.4, and Mixup/CutMix uses application probability 1 and switching probability 0.5. Learning rate warms up from 2×10^{-6} and decays to 2×10^{-5}. The cosine schedule uses the full 300-epoch time coordinate, with linear warmup replacing its first 20 epochs. There are 625 optimizer updates per epoch. Repeated augmentation uses timm's three-repeat convention, global sample permutation, rank-strided assignment, and a fixed epoch presentation budget. Persistent workers receive epoch-dependent random states. EMA updates after each optimizer step with constant decay 0.9999 from the first step. Each model trains for 300 epochs with seed 0.

Baseline reporting. Table 1 contains non-hierarchical models only. "iso." denotes isotropic ConvNeXt. The Type column uses SSM for state-space models, Linear for linear-attention and linear-recurrence mixers, and TTT for test-time training. ViT-B and Hydra-ViT-B use the EMA entries in Hydra's Table 5 [14]. Vim's † entries include 30 additional epochs of long-sequence fine-tuning at 224×224, with patch size 16 and extraction stride 8 [34]. LION's ♮ entries use multiple scans [1]. Parameter counts are rounded to whole millions and Top-1 to one decimal place. Training protocols differ across papers.

Convergence logs. Figure 4 uses the DeiT authors' four-GPU Small log. The authors state that this run differs from the released checkpoint. The log contains 300 epochs and ordinary-weight validation. Zero-based DeiT epochs are shifted by one to denote completed epochs. MemSolve-S uses the ordinary-weight validation recorded during the same run as its EMA result in Table 1. No smoothing, interpolation of missing epochs, or extension beyond a run's recorded training budget is applied. These runs illustrate recipe-level convergence; they do not isolate architecture or establish a training-time speedup.

C.2 Language pretraining

The completed encoder has 14 layers, width 768, twelve heads, rank 32, and 2,048 hidden units in each SwiGLU branch. Including the tied MLM head, it has 111,517,154 parameters; the encoder without that head has 110,894,504. WordPiece vocabulary size is 30,522. The learned absolute position table has 512 entries, and the token-type table has two. BERT embedding, residual-dropout, LayerNorm, and MLM-head conventions are retained: dropout 0.1, LayerNorm $\epsilon = 10^{-12}$, and a GELU/LayerNorm prediction transform with tied vocabulary decoder. Each centered depthwise Conv1d has width three and a residual connection; zero-initialized convolution weights make that connection initially the identity. Padding is masked before and after the local operation.

C4 training uses the complete local source shard collection through the Hugging Face Datasets streaming pipeline, with shuffle buffer 10,000. Each document is tokenized and truncated to 128 tokens, including special tokens. Sampling is budgeted by optimizer steps rather than a corpus pass. The budget is 286,720,000 sequence presentations, or 36.70 billion padded token slots; the latter is not a count of distinct or non-padding tokens. The MLM collator selects 30% of eligible training tokens and uses the standard 80/10/10 mask/random/unchanged corruption convention. Validation fixes the corruption of 10,000 C4 validation documents at 15% selection. Padding and special tokens are excluded from targets; `[MASK]` remains a valid context position. Validation NLL and accuracy are weighted by masked-target count.

We use fused PyTorch AdamW with betas $(0.9, 0.98)$, $\epsilon = 10^{-6}$, and gradient clipping at norm 1. The public LION recipe's Composer decay 10^{-5} is converted to PyTorch decay 0.0125 by division by the peak learning rate 8×10^{-4}. Bias and normalization parameters are excluded from pretraining decay. The run warms up for 4,200 steps and then decays linearly to 1.6×10^{-5}. This follows the public code's 2% endpoint; the LION paper's prose states 20%. Our document streaming, frozen validation subset, clipping, and HF optimizer execution also form part of the reported implementation.

Training uses two A800-SXM4-80GB GPUs, microbatch 512 per GPU, four accumulation steps, eight persistent data workers per rank, FP32 parameters, and BF16 autocast. Inductor compiles the surrounding model and CUDA Graph captures model/loss forward and backward. DDP communication and optimizer steps remain outside capture. The masked-target loss denominator spans all devices and accumulation steps. Validation and checkpointing occur every 1,000 updates. Minimum validation masked NLL selects step 70,000, which is also the final step. The recorded runtime is 114,490 seconds.

C.3 GLUE fine-tuning and repeated seeds

The classifier uses BERT's CLS pooler (dense plus $\tanh$), dropout, and a task-specific linear head. All encoder weights transfer from MLM; the absent pooler and classifier are initialized with BERT's initializer. For MNLI transfer, the trained pooler is retained and the classifier is reset. STS-B uses mean squared error; other tasks use cross-entropy.

Table 5: **GLUE fine-tuning configuration.** WD lists the source recipe value. For Composer rows, PyTorch WD equals the listed value divided by peak LR; STS-B uses its listed PyTorch value directly.

Task	Start	Epochs	Batch	Peak LR	WD	WD convention
MNLI	MLM	3	48	$5 \cdot 10^{-5}$	$5 \cdot 10^{-6}$	Composer
QNLI	MLM	10	16	$5 \cdot 10^{-5}$	10^{-5}	Composer
QQP	MLM	5	16	$3 \cdot 10^{-5}$	$3 \cdot 10^{-6}$	Composer
RTE	MNLI	3	16	10^{-5}	10^{-6}	Composer
SST-2	MLM	3	16	$3 \cdot 10^{-5}$	$3 \cdot 10^{-6}$	Composer
MRPC	MNLI	10	32	$8 \cdot 10^{-5}$	$8 \cdot 10^{-5}$	Composer
CoLA	MLM	10	32	$8 \cdot 10^{-5}$	$5 \cdot 10^{-6}$	Composer
STS-B	MNLI	10	32	$8 \cdot 10^{-5}$	$3 \cdot 10^{-6}$	PyTorch

All tasks use sequence length 128, AdamW betas $(0.9, 0.98)$, $\epsilon = 10^{-6}$, 6% step warmup, linear decay to zero, and no gradient clipping. Weight decay applies to all parameters in fine-tuning, as in the source GLUE jobs. Sequential sampling keeps partial final batches. Evaluation uses the final checkpoint on the development set; no hidden test labels are used. Fine-tuning uses eager execution of the BERT shell with the native CUDA mixer and BF16 autocast.

All runs share the seed-17 pretrained checkpoint. The MNLI-transfer tasks also share the seed-19 MNLI parent checkpoint. Their SD therefore measures variation in the final task's fine-tuning, not independent pretraining or independent MNLI transfer. The 81.9 aggregate averages the eight task means and has no suite-level SD because task seed counts differ.

Table 6: **MemSolve GLUE seed aggregation.** SD is the sample standard deviation over fine-tuning seeds. A dash denotes one run. Seed sets are $\mathcal{S}_1 = \{19\}$, $\mathcal{S}_3 = \{19, 8364, 717\}$, $\mathcal{S}_4 = \mathcal{S}_3 \cup \{10536\}$, and $\mathcal{S}_5 = \mathcal{S}_4 \cup \{90166\}$.

Task	Metric	Seeds	Mean	SD
MNLI	Matched/mismatched accuracy mean	$\mathcal{S}_1$	83.0	—
QQP	Accuracy/F1 mean	$\mathcal{S}_1$	89.8	—
QNLI	Accuracy	$\mathcal{S}_1$	88.6	—
SST-2	Accuracy	$\mathcal{S}_3$	91.6	0.350
CoLA	Matthews correlation	$\mathcal{S}_4$	47.5	1.505
RTE	Accuracy	$\mathcal{S}_5$	74.2	0.791
MRPC	F1	$\mathcal{S}_5$	91.0	0.653
STS-B	Spearman correlation	$\mathcal{S}_5$	89.2	0.140

C.4 Execution measurements and reproducibility

The sequence-parallel measurements use the archived October 16 implementation with the same equilibrium and complement equations. Each timing captures a complete layer forward and backward, including the sequence group's final shared-parameter gradient sum. It warms up five eager executions, captures a graph, warms up 20 replays, and records the median of three blocks of 50 replays with device synchronization. The distributed time is the maximum over both ranks. These measurements cover the layer; they do not include data loading or optimizer updates.

The training records preserve resolved recipes, source snapshots, data manifests, and checkpoint selection. The accompanying repository provides model code, training entrypoints, and figure-generation code at `https://github.com/Yang916-yy/MemSolve`. Figures are vector schematics of the equations and plots of all recorded epochs without smoothing. Scientific Agent Skills [15] provided writing and figure-review procedures for this revision; the draft remains subject to author verification.

D Extended Related Work

D.1 From memory updates to joint error correction

Linear attention accumulates feature–value associations in a compact state [16]. Delta-rule fast weights write the prediction residual instead, allowing each association to correct the current memory [23, 33]. GDNs add adaptive forgetting and richer erase/write controls [32, 17, 12]. TTT makes the internal-learning view explicit: the hidden state is a model trained on context [24]. Test-time regression connects several such update rules [29].

These connections motivate MemSolve's error-correction equation. The change is in how associations determine memory. Sequential updates evaluate each residual against the state reached so far. MemSolve evaluates all residuals against one shared state and solves their joint balance. Its construction therefore uses the full context without following an ordered update trajectory.

D.2 Regression layers and equilibrium models

Intention studies global regularized regression with query readout and feature-dimensional inversion [8]. MesaNet solves for locally optimal causal regression states using conjugate gradients [27]. ViT^3 studies full-batch inner updates for visual representations [11]. These works establish regression and internal optimization as ways to construct contextual memory.

MemSolve couples its write–read frame to the regularizer (Eq. (5)). This cancels the Gram term in the memory equation and leaves a learned core shared across samples. The reflected readout then controls the gain in token space. This coupling and readout distinguish the construction from a generic global regression layer.

Deep equilibrium and monotone equilibrium models also define states through equilibrium equations [2, 30]. MemSolve obtains its state from a compact linear system. Its contraction analysis uses the classical accretive–Cayley correspondence [3].

D.3 Bidirectional state-space and linear-recurrence models

Structured SSMs provide efficient sequence maps [10, 9, 5]. Vim adapts selective scans to vision, Hydra develops bidirectional matrix mixers, and LION relates full-attention, recurrent, and chunkwise forms [34, 14, 1]. Video DeltaNet combines local attention with bidirectional memory updated jointly within each frame [31].

MemSolve gathers full-input statistics before solving one memory. Its frame normalization couples information from both sides of a token. Theorem 3.3 shows that locally parameterized bidirectional scans with linear merging cannot express all of these interactions in one mixer. The distinction concerns input dependence as well as execution: the compact solve can use full-context statistics while retaining linear complexity in sequence length at fixed rank.